LAB 3 REPORT

Monitoring, Logging & DevSecOps

Lab 3: Monitoring, Logging & DevSecOps | CO2, CO3
Tools: Ansible, Terraform, Prometheus, Grafana, ELK, Jenkins, Selenium, JMeter
App: sample-app | Infra: 2 EC2 / Local VMs - web + monitoring

Exp 1 (5 marks): Ansible - Automate Config

Aim: Playbook writes config to target nodes.

inventory.ini:
[web]
192.168.1.10 ansible_user=ubuntu

playbook.yml:
- hosts: web

```
  become: yes
  tasks:
  - name: Install nginx
    apt: name=nginx state=present
  - name: Copy index.html
    copy: src=index.html dest=/var/www/html/
  - name: Start nginx
    service: name=nginx state=started
```

Commands:

```
$ ansible --version
$ ansible-playbook -i inventory.ini playbook.yml
$ ansible web -m ping
$ ansible-playbook --check playbook.yml
```

Result: nginx installed on target, site live.
Observation: Idempotent - 2nd run shows 0 changed.
Screenshot: Playbook run SUCCESS + nginx page

Exp 2 (5 marks): Terraform - Provision Scalable Infra

main.tf:

```
provider "aws" { region = "ap-south-1" }
resource "aws_instance" "web" {
  ami = "ami-0abcdef"
  instance_type = "t2.micro"
  count = 2
  tags = { Name = "web-server" }
}
```

Commands:

```
$ terraform init
$ terraform plan
$ terraform apply -auto-approve
$ terraform show
$ terraform destroy -auto-approve
```

Result: 2 EC2 created, IPs in output.
Observation: IaC reproducible, destroy cleans all.

LAB 3 REPORT

Monitoring & Logging

Exp 3 (5 marks): Monitoring - Prometheus + Grafana

prometheus.yml:

global: {scrape_interval: 15s}

```
scrape_configs:
- job_name: 'node'
  static_configs:
  - targets: ['localhost:9100']
- job_name: 'app'
  static_configs:
  - targets: ['localhost:3000']
```

Commands:

```
$ docker run -d -p 9090:9090 prom/prometheus
$ docker run -d -p 9100:9100 prom/node-exporter
$ docker run -d -p 3000:3000 grafana/grafana
$ curl localhost:9090/metrics
$ curl localhost:9100/metrics
```

Grafana: Add Data Source Prometheus -> http://localhost:9090

Dashboard: Import ID 1860 - Node Exporter Full

Panels: CPU %, Memory %, Disk, Network, HTTP Requests

Observation: CPU spike visible live, alert on CPU > 80%.

Screenshot: Prometheus targets UP + Grafana dashboard

Exp 4 (5 marks): Centralized Logging - ELK Stack

docker-compose.yml: Elasticsearch 9200, Logstash 5044, Kibana 5601

logstash.conf:

```
input { beats { port => 5044 } }
filter { grok { match => { message => "%{COMBINEDAPACHELOG}" } } }
output { elasticsearch { hosts => ["elasticsearch:9200"] } }
```

Commands:

```
$ docker compose up -d
$ curl localhost:9200
$ curl localhost:5601
```

Filebeat: filebeat.yml -> output.logstash hosts: ['localhost:5044']

```
$ filebeat -e -c filebeat.yml
```

Kibana: Discover -> Create Index Pattern logstash-* -> Visualize
errors

Query: status:500 -> 12 errors found, status:200 -> 1450 hits

Observation: All logs in one place, search in seconds.

Screenshot: Kibana Discover + Visualization

LAB 3 REPORT

DevSecOps, Flags & Testing

Exp 5 (5 marks): DevSecOps - Security in CI/CD

Pipeline with scanning:

pipeline {

```
  stages {
    stage('Scan') { sh 'trivy image myapp:v1' }
    stage('Dependency') { sh 'npm audit' }
    stage('SAST') { sh 'sonar-scanner' }
  }
```

}

Commands:

```
$ trivy image myapp:v1
$ npm audit
$ npm audit fix
$ owasp-dependency-check --scan .
$ sonar-scanner
```

Findings: Trivy - 2 HIGH, 1 CRITICAL in base image
Fix: FROM node:18-alpine -> node:20-alpine, npm audit fix
After fix: 0 CRITICAL, 0 HIGH, Pipeline SUCCESS
Secure Practices: Secrets in Jenkins Credentials, no hardcode,
least privilege, image signing.

Observation: Shift-left - bug caught before deploy.
Screenshot: Trivy report before/after + Jenkins scan stage

Exp 6 (5 marks): Feature Flags, Load Testing, Selenium

Feature Flag - config.json:
{ "newCheckout": true, "betaUI": false }
app.js: if (config.newCheckout) showNew() else showOld()
Toggle true -> New UI, false -> Old UI, no redeploy.

Load Testing - JMeter:
Thread Group: 100 users, Ramp 10s, Loop 10

```
$ jmeter -n -t load.jmx -l result.jtl
```

Result: Avg 210ms, Throughput 420 req/sec, Error 0.5%
At 500 users: Avg 850ms, Error 3% - Bottleneck found.

Selenium - test.py:

```
from selenium import webdriver
driver = webdriver.Chrome()
driver.get('http://localhost:3000')
assert 'Hello' in driver.title
driver.find_element('id','login').click()
print('Login Test PASS')
```

Commands:

```
$ python test.py -> 3/3 tests PASS
```

Observation: Flags allow safe rollout, load test shows limit.

LAB 3 REPORT

Final Report - Monitoring & DevSecOps

Exp 7 (10 marks): Monitoring, Logging & DevSecOps Report

Architecture:

Ansible + Terraform -> Infra -> App -> Prometheus + Grafana
-> Metrics | App Logs -> Filebeat -> Logstash -> Elasticsearch ->
Kibana
-> CI/CD -> Trivy + npm audit + Sonar -> Secure Deploy
-> Feature Flags -> JMeter + Selenium -> Release

Automation Results:

Infra Provision: Terraform 2 VMs in 90 sec
Config: Ansible 0 manual steps, 100% idempotent
Monitoring: 15s scrape, Grafana 6 panels live
Logging: 1500+ logs centralized, search <2 sec
Security: 3 Critical fixed before deploy
Testing: Selenium 3/3 PASS, JMeter 420 req/sec

Observations:

- Observability reduces MTTR from hours to minutes
- Central logs make debugging easy
- DevSecOps prevents vulnerable deploy
- Feature flags enable safe rollback in 1 click

Conclusion: Automated, observable, secure DevOps workflow achieved.